CSEN/CSIS402 — Computer Organization

Milestone 2 Report: Control Unit Implementation

German University in Cairo — Spring Term 2026
Dr. Milad Ghantous — Dr. Shereen Afifi

Team 50

Name	Email	ID	Tutorial
Habeeba Essam	habeeba.fahim@student.guc.edu.eg	64-3229	T-15
Salma Ahmed	salma.helaly@student.guc.edu.eg	64-13563	T-22
Mariam Mostafa	Mariam.Hamouda@student.guc.edu.eg	64-24804	T-15
Mariam Ahmed	mariam.niazi@student.guc.edu.eg	64-9453	T-10
Mariam Hamama	maryam.elket@student.guc.edu.eg	64-6808	T-9
Haya Mohamad	haya.ragab@student.guc.edu.eg	64-10924	T-8

1. Program Description

The following C program was implemented in assembly and loaded into the basic computer:

```
int A = 15, B = 5, C = 0, i = -3;
for (i = 0; i < 3; i++) {
    A += A XOR B;
    C += A / B;
}
// Expected output: A = 101, C = 35
```

The program uses 7 instructions: LDA, STA, ADD, DIV, XOR, ISZ, BUN.

2. Opcode Assignment

Instruction opcodes are encoded in bits 15–12 of each 16-bit instruction word. The address field occupies bits 11–4. Bits 3–0 are unused.

Decoder Output	Instruction	Opcode (bits 15–12)	Description
D0	LDA	0000 → repurposed	Load DR into AC

Decoder Output	Instruction	Opcode (bits 15–12)	Description
D1	STA	0001 → repurposed	Store AC into memory
D2	ADD	0010	AC ← AC + DR
D3	DIV	0011 → repurposed (AND)	AC ← AC / DR
D4	XOR	0100 → repurposed (BSA)	AC ← AC XOR DR
D5	ISZ	0101	Increment and skip if zero
D6	BUN	0110	Branch unconditional

3. Memory Contents

Program loaded at ORG 0. Data variables follow at addresses 0x0B–0x0E.

Address	Label	Instruction	Hex Value
00	LOP	LDA A	2B00
01		XOR B	4C00
02		ADD A	1B00
03		STA A	3B00
04		LDA A	2B00
05		DIV B	0C00
06		ADD C	1D00
07		STA C	3D00
08		ISZ i	5E00
09		BUN LOP	6000
0A		HLT (ignored)	7000
0B	A	DEC 15	000F
0C	B	DEC 5	0005
0D	C	DEC 0	0000
0E	i	DEC -3	FFFD

4. Instruction RTL (Register Transfer Language)

4.1 Fetch Cycle — Common to All Instructions

T0: AR ← PC
T1: IR ← M[AR], PC ← PC + 1
T2: AR ← IR[4–11], Decode IR[12–15]

4.2 LDA — Load Accumulator (D0)

T3: DR ← M[AR]
T4: AC ← DR, SC ← 0

4.3 STA — Store Accumulator (D1)

T3: M[AR] ← AC, SC ← 0

4.4 ADD — Add to Accumulator (D2)

T3: DR ← M[AR]
T4: AC ← AC + DR, SC ← 0

4.5 DIV — Divide Accumulator (D3)

T3: DR ← M[AR]
T4: AC ← AC / DR, SC ← 0

4.6 XOR — Logical XOR (D4)

T3: DR ← M[AR]
T4: AC ← AC XOR DR, SC ← 0

4.7 ISZ — Increment and Skip if Zero (D5)

T3: DR ← M[AR]
T4: DR ← DR + 1
T5: M[AR] ← DR
T6: if (DR = 0) then PC ← PC + 1, SC ← 0

4.8 BUN — Branch Unconditional (D6)

T3: PC ← AR, SC ← 0

5. Control Signal Boolean Expressions

5.1 Sequence Counter (SC) Control

CLR_SC = T4(D0 + D2 + D3 + D4) + T3(D1 + D6) + D5·T6

The SC clears at the last time step of each instruction:

Instruction	Clears At	Term in CLR_SC
LDA (D0)	T4	T4·D0
STA (D1)	T3	T3·D1
ADD (D2)	T4	T4·D2
DIV (D3)	T4	T4·D3
XOR (D4)	T4	T4·D4
ISZ (D5)	T6	T6·D5
BUN (D6)	T3	T3·D6

5.2 Register Load (LD) Signals

Signal	Boolean Expression
LD_AR	T0 + T2
LD_IR	T1
LD_PC	T3·D6
LD_DR	T3·(D0 + D2 + D3 + D4 + D5)
LD_AC	T4·(D0 + D2 + D3 + D4)

5.3 Register Increment (INR) Signals

Signal	Boolean Expression
INR_PC	T1 + (D5·T6·Z) where Z = 1 when DR = 0
INR_DR	D5·T4
INR_AR	0 (not used)
INR_AC	0 (not used)
INR_IR	0 (not used)

5.4 Memory Control Signals

Signal	Boolean Expression
MEM_READ	T1 + T3·(D0 + D2 + D3 + D4 + D5)
MEM_WRITE	T3·D1 + T5·D5

6. Bus Select Lines — Priority Encoder

The common bus uses a 7-to-3 priority encoder. Each encoder input corresponds to one bus source. The encoder output directly drives S2S1S0. I0 is left unconnected (000 = idle, no source drives the bus).

Encoder Input	S2S1S0	Bus Source	Boolean Expression
I0	000	Idle (no source)	— (left unconnected)
I1	001	Memory output	T1 + T3·(D0+D2+D3+D4+D5)
I2	010	AR output	0 (not used)
I3	011	PC output	T0
I4	100	DR output	T4·(D0+D2+D3+D4) + T5·D5
I5	101	AC output	T3·D1
I6	110	IR[4-11] output	T2
I7	111	TR output	0 (not used)

In Logisim: build gate networks for I1, I3, I4, I5, I6 only. Feed all 8 lines into a priority encoder. Connect 3-bit output to S2, S1, S0 tunnels. I0, I2, I7 tied to constant 0.

7. ALU Select Lines — Priority Encoder

The ALU uses a 7-to-3 priority encoder based on the project ALU table. Each encoder input corresponds to one ALU operation. I0 is left unconnected (000 = idle).

Encoder Input	C2C1C0	Operation	Boolean Expression
I0	000	Idle	— (left unconnected)
I1	001	Transfer A (pass DR)	T4·D0 (LDA)
I2	010	Add A + B	T4·D2 (ADD)

Encoder Input	C2C1C0	Operation	Boolean Expression
I3	011	Subtract A - B	0 (not used)
I4	100	Multiply A x B	0 (not used)
I5	101	Divide A / B	T4·D3 (DIV)
I6	110	Complement A	0 (not used)
I7	111	A XOR B	T4·D4 (XOR)

In Logisim: build AND gates for I1 (T4·D0), I2 (T4·D2), I5 (T4·D3), I7 (T4·D4). Feed all 8 lines into a priority encoder. Connect 3-bit output to C2, C1, C0 tunnels. I3, I4, I6 tied to constant 0.

8. Implementation Notes

1. IR is implemented as a Register (not a counter) per project specification.
2. PC is implemented as a counter to allow direct increment via INR pin.
3. The CLR signal on all counter registers is asynchronous in Logisim. A D Flip-Flop is placed between the CLR logic output and the register CLR pin to enforce synchronous clearing on the clock edge.
4. The DRZERO flag (Z) is implemented as a 16-input OR gate on DR's output — if all bits are 0, the OR output is 0, which after a NOT gate gives Z = 1.
5. Tunnels are used throughout to avoid crossed wires. Tunnels do not work across circuits — all control logic is within the same circuit.
6. DIV uses the AND opcode slot (repurposed as D3 = 0011) and XOR uses the BSA opcode slot (repurposed as D4 = 0100), following the D mapping: D0=LDA, D1=STA, D2=ADD, D3=DIV, D4=XOR, D5=ISZ, D6=BUN.